

Cricket Ball Detection & Trajectory Tracking System

1. Problem Statement

The objective of this project is to **detect a cricket ball in a video**, **track its movement across frames**, and **store its spatial coordinates** in a structured format. Additionally, the system should generate an **annotated output video** that visually validates detection quality and illustrates the **trajectory of the ball** over time.

Key challenges include:

- Very **small object size**
- **High-speed motion** leading to blur
- **Intermittent occlusion** (bat, player, pitch)
- Maintaining detection consistency across frames

2. Model Selection & Design Decisions

2.1 Choice of Model: YOLOv11

YOLOv11 was selected as the primary model due to:

- Real-time inference capability
- Strong performance on small object detection
- Simple training pipeline
- Robust PyTorch ecosystem support

The **YOLOv11n (nano)** variant was chosen initially to:

- Reduce training time
- Enable faster experimentation
- Fit Google Colab GPU constraints

Once stability was confirmed, training settings were optimized to improve accuracy.

2.2 Single-Class Training Strategy

Instead of multi-class detection, the model was trained on **only one class**:

Class 0 $\rightarrow$ cricket_ball

Rationale:

- Reduces class confusion
- Improves precision for a small object
- Simplifies annotation and evaluation

3. Dataset Preparation & Assumptions

3.1 Dataset Structure

- The dataset follows the standard YOLO format:
- Train/images, valid/images
- Train/labels, valid/labels
- data.yaml
- Each image has a corresponding .txt annotation file.

3.2 Annotation Assumptions

- Only **one cricket ball** appears per frame
- Bounding boxes tightly enclose the ball
- Background clutter is not annotated
- Frames without a visible ball have **no bounding box**

3.3 Coordinate Representation

- Bounding boxes are converted to **center coordinates (x, y)**
- Coordinates are stored in **pixel space**
- No camera calibration or real-world scaling is assumed

4. Inference Pipeline & Fallback Logic

4.1 Frame-wise Detection

For each video frame:

- YOLO predicts bounding boxes
- The center (x, y) is computed
- Confidence score is evaluated

4.2 Visibility Flag (Fallback Logic)

A **visibility flag** is introduced to handle missed detections:

Value	Meaning
1	Ball detected
0	Ball not detected

Fallback behavior:

- If no detection occurs in a frame:
 - (x, y) is stored as None
 - visible = 0
- This preserves frame alignment and allows post-processing

4.3 Why This Matters

- Real-world videos contain occlusions
- Detection models are not perfect
- Visibility flags enable:
 - Interpolation
 - Smoothing
 - Error analysis

5. Trajectory Overlay Logic

5.1 Concept

The **trajectory overlay** visualizes the path of the ball by:

- Storing historical center points
- Drawing line segments between consecutive positions

5.2 Implementation Details

- A rolling buffer stores recent (x, y) point
- Old points are discarded after a threshold
- Lines are drawn only when detections are valid

This results in a smooth, interpretable motion trail.

6. Output Artifacts

6.1 CSV Output

The system generates a CSV file with the format:

frame_id, x, y, visible

Example:

12, 418.2, 290.1, 1

13, , , 0

14, 425.8, 296.4, 1

6.2 Annotated Video

The output video includes:

- Bounding box around the ball
- Label: ball
- Trajectory path overlay

This provides **visual validation** of detection accuracy.

7. Limitations & Assumptions

- Pixel coordinates only (no real-world mapping)
- Single ball assumption
- No physics-based smoothing

- Performance depends on video quality

11. Conclusion

This project demonstrates a **robust, end-to-end cricket ball detection and tracking system** using YOLOv11. Careful modelling decisions, fallback logic, and iterative improvements resulted in a reliable pipeline capable of handling real-world video challenges. The system is suitable for **sports analytics, research prototypes, and technical assessments**.